10 Attention Mechanisms and Transformers

Chapter Abstract. This chapter introduces attention mechanisms and Transformers as practical tools for sequence and text problems. The mathematical running example is machine translation: when a decoder produces the wrong noun, the useful question is whether it could look back at the right source token. That example motivates the fixed-vector bottleneck in early encoder–decoder models, additive and multiplicative attention, RNNs with attention, CNNs with attention, self-attention, and the Transformer architecture. The applied running example is multi-label toxic-comment classification, where a pretrained Transformer encoder is fine-tuned with the Hugging Face Transformers library. The chapter emphasizes rigorous definitions, worked numerical examples, masks, shape checks, practical choices that affect accuracy and runtime, and honest validation practice.

10.1 From Recurrent Networks to Attention

The recurrent neural networks in Chapter 9 introduced a central idea for sequence data: the order of the inputs matters. A movie review, a sentence, an audio recording, and a stream of sensor readings are not ordinary unordered feature vectors. In an RNN, the model carries a hidden state through the sequence, updating that state one step at a time. This mechanism is useful and historically important, but many applied problems need a model that maps one structured object into another structured object. A translation system reads a source sentence and writes a target sentence. An image-captioning system reads an image and writes a sentence. A speech recognizer reads acoustic features and writes characters or words. These are not only sequence-classification problems; they are input-output mapping problems in which the output length may differ from the input length.

The mathematical running example in this chapter is machine translation. A short English sentence such as "the movie was not good" might be translated into another language with a different word order and a different number of tokens. The source sentence will be denoted

$$x_{1:T_x} = (x_1, x_2, \ldots, x_{T_x}),$$

and the target sentence will be denoted

$$y_{1:T_y} = (y_1, y_2, \ldots, y_{T_y}).$$

Here T_x and T_y are lengths, not fixed constants shared by every example. In a real data pipeline, the symbols x_i and y_t usually begin as token IDs produced by a tokenizer, then become vectors through an embedding layer.

The chapter also keeps a practical training example in view: toxic-comment classification. In the Jigsaw Toxic Comment Classification Challenge, each comment may have several labels at once, such as toxic, severe toxic, obscene, threat, insult, and identity hate [120]. This is a multi-label classification problem, not a translation problem, but it is where pretrained Transformer encoders are often useful in applied work. A model such as BERT or ModernBERT can read the whole comment and produce a representation that a classification head turns into six label logits. The training question then becomes practical: which checkpoint, sequence length, batch size, learning rate, metric, and validation protocol give a strong result within a reasonable runtime?

The common thread is selective use of information. A translation decoder may need the word "not" at the exact step where it produces a target expression for negation. An image-captioning decoder may need the region containing a bicycle only when the caption reaches that object. A toxic-comment classifier has a different output format, but it still has to combine local phrases, long-range context, and punctuation without letting padding tokens become evidence. Attention is the mechanism that makes this selective use explicit and differentiable.

Attention is selective retrieval. The model learns which positions to read for the current prediction instead of compressing everything into one undifferentiated vector.

10.2 Encoder–Decoder Architecture

An encoder–decoder model separates an input-output problem into two parts. The encoder reads the input and produces a representation. The decoder uses that representation, together with any output already produced, to produce the next output. This pattern became central in neural machine translation because the input and output are both sequences but need not have the same length [268, 28]. It is also useful outside text: an image-captioning model may use a CNN encoder for the image and an RNN decoder for the caption [293]; an end-to-end speech recognizer may use an acoustic encoder and an attention-based character decoder [23].

Definition 10.2.1 (Encoder, decoder, and encoder–decoder model) *An* encoder *is a learned function that maps an input object to one or more internal representations. A* decoder *is a learned function that maps those representations, and often the previously generated outputs, to the next output. An* encoder–decoder model *combines these two functions to model an input-output conditional distribution.*

In the simplest sequence-to-sequence notation, the encoder maps a source sequence to one vector,

$$z = f_\theta(x_{1:T_x}) \in \mathbb{R}^d,$$

and the decoder predicts the target sequence one token at a time:

$$p_\phi(y_{1:T_y} \mid x_{1:T_x}) = \prod_{t=1}^{T_y} p_\phi(y_t \mid y_{<t}, z).$$

Table 10.1: Encoder–decoder thinking appears in several applied tasks whose outputs are generated as sequences.

Application	Input	Encoder	Decoder output
Machine translation	Source tokens	Text sequence encoder	Target tokens
Image captioning	Image tensor	CNN or vision encoder	Caption tokens
Speech recognition	Acoustic frames	Audio sequence encoder	Character or word tokens

The expression $y_{<t}$ means the target tokens before position t. During training, those previous target tokens are usually the correct previous tokens from the dataset. This procedure is called teacher forcing. During inference, the decoder does not have the true target sentence, so it conditions on tokens that it has already generated. This difference is one reason sequence generation is more delicate than ordinary classification.

Teacher forcing is training-only. At inference time, decoding depends on the model's own earlier predictions, so errors can affect later tokens.

The notation is compact, but the debugging question is concrete: if the fifth generated token is wrong, was the decoder missing the needed source evidence, or did it have the evidence and use it badly?

Image captioning is the same pattern with a different encoder. A CNN can transform an image into a feature vector z_{image}, and a language decoder can generate a caption:

$$p(y_t \mid y_{<t}, z_{\text{image}}).$$

Speech recognition uses the pattern again, but with an acoustic encoder that reads short-time audio features and a decoder that emits characters, subword tokens, or words. The preprocessing and metrics differ, but the structural question is shared: how should the decoder obtain the information it needs from the encoder?

Table 10.1 is deliberately practical. A model family does not by itself determine success. Image captioning needs image normalization and caption tokenization. Speech recognition needs carefully computed acoustic features or a model that learns them. Translation needs tokenization, maximum lengths, and a decoding rule such as greedy decoding or beam search.

The toxic-comment running example in this chapter is related but not identical. It is usually an encoder-plus-classification-head problem rather than an encoder–decoder generation problem. The encoder reads the whole comment, and a prediction head produces six label logits. This distinction matters in practice: toxic-comment classification needs a multi-label loss, a metric appropriate for imbalanced labels, and a validation split that is not repeatedly reused as if it were a test set; it does not need a token-by-token decoder.

Match architecture to task. Classification needs label logits; translation and summarization need generated token sequences.

Pause and predict

Suppose an encoder reads a source sentence with $T_x = 7$ tokens and produces one vector $z \in \mathbb{R}^{128}$. A decoder then generates $T_y = 5$ target tokens. Before reading on, identify which quantity has fixed dimension and which quantities may change from example to example.

The vector z has fixed dimension 128, while T_x and T_y may change across examples.

10.3 The Fixed-Vector Bottleneck

The earliest neural encoder–decoder translation systems passed information from the encoder to the decoder through one fixed-length vector. In an RNN encoder, this vector was often the final hidden state. If the source sentence was $x_1, \ldots, x_{T_x}$, the encoder produced hidden states

$$h_i = \mathrm{RNN}_{enc}(x_i, h_{i-1}),$$

and the decoder received only $z = h_{T_x}$. The decoder then had to generate every target token from this same vector. This design is mathematically simple, but it asks one vector to carry every source detail that might become useful at any later target position.

Bottleneck means reuse pressure. One fixed vector must support every output step, even when different steps need different source details.

The limitation is easiest to see with a concrete translation. Consider the source sentence "the movie was not good." A target language may place the negation marker near a different word or may express "not good" with a single word. When the decoder produces the target word corresponding to "movie," it needs one part of the source. When it produces the target expression for negation, it needs another part. A single vector z gives the decoder no explicit operation for saying "look again at the fourth source token now." The vector may still contain useful information, especially for short sentences, but the connection is indirect.

Pause and predict

Suppose an encoder keeps only $z = h_{T_x}$ for a six-token source sentence, and the next target word depends mainly on token x_2. Predict why making z wider may help, but still does not give the decoder a direct operation for choosing token x_2 at that output step.

Bahdanau, Cho, and Bengio described this as a fixed-length vector bottleneck and proposed that the decoder should be allowed to search, softly and differentiably, over the source annotations while generating each target word [7]. The word "bottleneck" should be read carefully. It does not mean that a fixed vector can never work. It means that the same fixed vector must serve all output positions, so the model has a harder optimization problem as sentences become longer, word order becomes more complex, or localized details become important.

A small numerical example makes the compression issue visible. Suppose four encoder states are

$$h_1 = (1, 0), \qquad h_2 = (0, 1), \qquad h_3 = (1, 1), \qquad h_4 = (0, -1).$$

A fixed-vector encoder might pass only $z = h_4 = (0, -1)$ to the decoder. If the next target word depends mainly on the first source token, the decoder has no direct access to $h_1 = (1, 0)$. Another model might pass an average

$$\bar{h} = \frac{1}{4}(h_1 + h_2 + h_3 + h_4) = (0.5, 0.25),$$

which preserves some global information but still blurs the four positions together. Attention keeps the individual states available and lets the decoder form a different weighted combination at each target step.

Figure 10.1 puts this contrast on one page. The important change is not just adding more parameters; it is giving the decoder an explicit operation for rereading different source positions at different target steps.

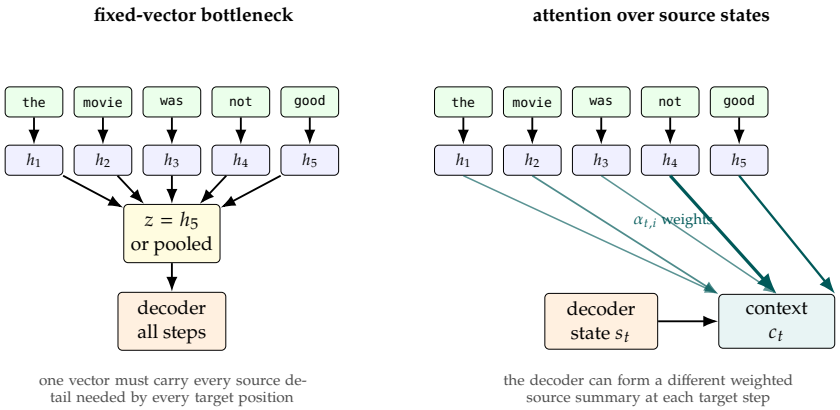


Figure 10.1: Fixed-vector sequence-to-sequence models compress the source into one representation, while attention lets each decoder step retrieve a task-specific mixture of encoder states.

The same pressure appears outside translation. One global image vector may be enough to predict that an image is about a street scene, but the phrase "traffic light" depends on a particular region. One global vector for a whole utterance would make it difficult to align acoustic evidence with emitted characters. A toxic-comment classifier can use a pooled representation at the end, but modern encoders make that pooled vector stronger by first letting token representations exchange information through self-attention.

10.4 The Attention Model

Attention can be described as learned retrieval. A query represents the information currently being requested. Keys represent the locations that can be searched. Values represent the information that will be returned. The model compares the query with each key, converts the comparison scores into nonnegative weights, and returns a weighted average of the values. Unlike a hard database lookup, this operation is differentiable, so it can be trained by backpropagation as part of a neural network.

Q/K/V vocabulary. The query asks what is needed, keys decide where to look, and values provide what is returned.

This vocabulary prevents a common debugging mistake: treating attention as a vague explanation instead of asking which representation made the request, which positions were searchable, and which information was actually mixed into the output.

Definition 10.4.1 (Attention) *Let $q \in \mathbb{R}^{d_q}$ be a query, let $k_i \in \mathbb{R}^{d_k}$ be keys, and let $v_i \in \mathbb{R}^{d_v}$ be values for $i = 1, \ldots, n$. An attention mechanism computes scores*

$$e_i = \text{score}(q, k_i),$$

attention weights

$$\alpha_i = \frac{\exp(e_i)}{\sum_{j=1}^{n} \exp(e_j)},$$

and a context vector

$$c = \sum_{i=1}^{n} \alpha_i v_i.$$

The weights satisfy $\alpha_i \geq 0$ and $\sum_i \alpha_i = 1$.

Figure 10.2 shows the same definition as a computation pattern. The scores decide where to read, the softmax turns those scores into a probability-like distribution, and the values are the information being mixed.

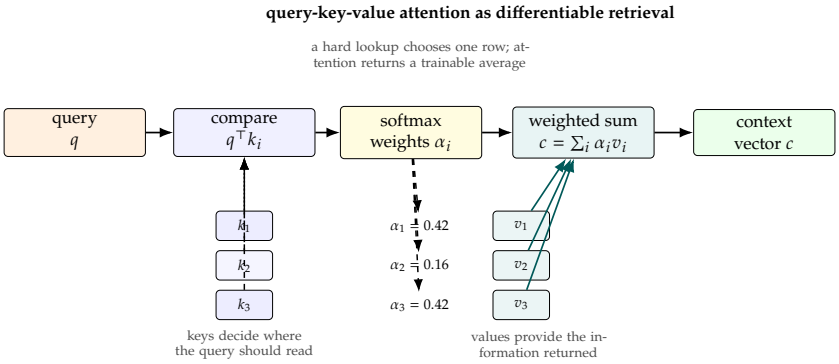


Figure 10.2: The query is compared with keys to produce attention weights; those weights average the values into the context vector used by the next part of the network.

In the translation running example, the decoder state at target position t can be the query. The encoder states for the source words can be the keys and values. If the decoder is about to produce the target word corresponding to "movie," the attention weights should ideally put more mass on the source position containing "movie." If the decoder is about to express the negation in "not good," the weights may move toward the source position containing "not" and the nearby adjective.

This is the practical value of attention in an applied model: when the output says the wrong thing, the first inspection is often whether the model looked in the right place before it spoke.

A complete attention calculation can be done by hand with small vectors. Let

$$q = (1, 0),$$

and let the three keys be

$$k_1 = (1, 0), \qquad k_2 = (0, 1), \qquad k_3 = (1, 1).$$

Using dot-product scores $e_i = q^\top k_i$, the scores are

$$e_1 = 1, \qquad e_2 = 0, \qquad e_3 = 1.$$

The exponentials are approximately

$$\exp(e_1) = 2.718, \qquad \exp(e_2) = 1.000, \qquad \exp(e_3) = 2.718,$$

with total 6.436. Therefore

$$\alpha_1 \approx 0.422, \qquad \alpha_2 \approx 0.155, \qquad \alpha_3 \approx 0.422.$$

If the corresponding values are

$$v_1 = (2, 0), \qquad v_2 = (0, 3), \qquad v_3 = (1, 1),$$

then the context vector is

$$\begin{aligned} c &= 0.422(2, 0) + 0.155(0, 3) + 0.422(1, 1) \\ &\approx (1.266, 0.887). \end{aligned}$$

This small example shows the essential behavior. The query matches k_1 and k_3 equally well, matches k_2 less well, and therefore returns a context vector dominated by v_1 and v_3 but not exactly equal to either one.

Two score functions are especially important historically. Additive attention, introduced in neural machine translation by Bahdanau and colleagues, scores a query-key pair with a small neural network [7]:

$$e_i = w_a^\top \tanh(W_q q + W_k k_i + b_a).$$

Here W_q, W_k, w_a, and b_a are learned parameters. Additive attention can learn a flexible compatibility function even when the query and key dimensions differ. It is conceptually close to the feedforward networks used in earlier chapters: concatenate or combine features, transform them through a nonlinearity, and produce a scalar score.

Multiplicative attention scores a query-key pair with a product. Luong, Pham, and Manning studied several effective attention variants for neural machine translation [173]. The simplest dot-product score is

$$e_i = q^\top k_i,$$

and a learned bilinear version is

$$e_i = q^\top W_a k_i.$$

Dot-product attention is less flexible than a general small neural network, but it is especially efficient on modern hardware because all scores can be computed as matrix multiplications. This computational advantage becomes central in Transformers.

Vaswani and colleagues used scaled dot-product attention in the Transformer [292]. If the query and key dimension is d_k, the score matrix is divided by $\sqrt{d_k}$:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

The scaling prevents dot products from becoming too large in magnitude as the dimension grows. Large positive or negative scores can make the softmax nearly one-hot, which may produce small gradients early in training.

Scale before softmax. Dividing by $\sqrt{d_k}$ keeps dot-product scores from becoming too sharp just because vectors are wider.

Masks are part of the definition in real models. Suppose a model has scores

$$(2, 1, -4, 0)$$

for four positions, but the fourth position is padding. Before the softmax, the padding score is replaced by $-\infty$, so the effective scores are

$$(2, 1, -4, -\infty).$$

The exponentials are approximately $(7.389, 2.718, 0.018, 0)$, so the weights are approximately

$$(0.730, 0.268, 0.002, 0).$$

The padding position receives exactly zero attention weight. Without this mask, a model could learn from meaningless padding tokens, and a reported validation score might depend on batch padding rather than on the actual examples.

Masks are model inputs. Padding and causal masks change which positions are visible, so they are part of the computation, not display settings.

Pause and predict

In the dot-product example above, what would happen to the attention weights if the query changed from $q = (1, 0)$ to $q = (0, 1)$?

The second key would now match strongly, so the context vector would move toward the value v_2.

10.5 RNN Encoder–Decoders With Attention

Attention first became widely known in neural machine translation as an extension of RNN encoder–decoder models. The RNN encoder still reads the source sentence in order and produces a sequence of hidden states

$$h_i = \text{RNN}_{enc}(x_i, h_{i-1}), \qquad i = 1, \ldots, T_x.$$

In many systems, those encoder states come from a bidirectional recurrent encoder: one recurrence reads left to right, another reads right to left, and their states may be concatenated. The decoder is still autoregressive; bidirectionality belongs on the source encoder side, where the whole source sentence is already known. The difference is that the decoder no longer receives only a single vector. It receives access to all encoder states $h_1, \dots, h_{T_x}$. At each target position, the decoder state asks a new question of the source sequence.

Let s_t denote the decoder hidden state at target position t. A common attention-based decoder has the form

$$c_t = \text{Attention}(q = s_t,\ K = H,\ V = H),$$

where H is the matrix whose rows are the encoder states. The context vector c_t is then combined with the decoder state to predict the next target token:

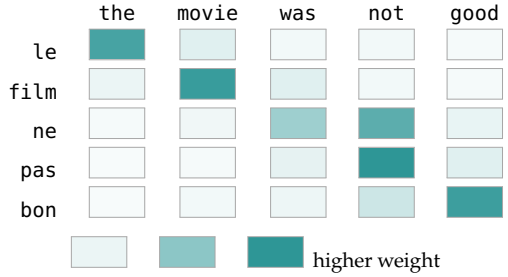
$$p(y_t \mid y_{<t}, x_{1:T_x}) = \text{softmax}(W_o[s_t; c_t] + b_o).$$

The notation $[s_t; c_t]$ means concatenation. In words, the model predicts the next token from both the decoder's current memory and a source-dependent context vector chosen for this particular output step.

This construction gives the RNN decoder a more direct alignment mechanism. For the source sentence "the movie was not good," the attention weights at one target step may concentrate on "movie," while the weights at a later step may concentrate on "not" and "good." The model is not given the correct alignment by hand. It learns attention weights because those weights help reduce the translation loss. This is why the mechanism is often called soft alignment: every source position can receive some weight, but the distribution can become sharply concentrated when the task benefits from that behavior.

Alignment is learned. Attention weights are trained through the task loss; they are not supervised word-alignment labels.

Figure 10.3 shows this idea as an alignment heatmap. The rows are decoder steps and the columns are source positions. The figure is illustrative, not a measured translation run, but it shows the diagnostic reading: if the model produces a word expressing negation, the row for that decoder step should often place substantial weight on the source word that carried the negation.



Each row is one generated target token; each column is one source token. Dark cells show source positions that the decoder reads most strongly at that step.

Figure 10.3: Illustrative RNN attention alignment for a short translation example. The weights are learned through the sequence loss and need not be supervised word alignments, but a concentrated pattern can reveal which source tokens each decoder step is using.

Pause and predict

Suppose $T_x = 6$, the encoder states are stored in $H_{enc} \in \mathbb{R}^{6\times 8}$, and the decoder query is $q_t \in \mathbb{R}^8$. Before reading the shape check, predict the shape of the attention scores and the shape of the context vector c_t.

The shape check is simple but important. Suppose a source sentence has $T_x = 6$ tokens and the encoder hidden width is $H = 8$. The encoder states can be stored in a matrix $H_{enc} \in \mathbb{R}^{6\times 8}$. If the decoder state has the same width, then a dot-product attention step uses a query $q_t \in \mathbb{R}^8$. Multiplying the matrix by the query gives six scores, one per source token. After softmax, the six weights multiply the six encoder states to produce a context vector $c_t \in \mathbb{R}^8$. This shape reasoning catches many implementation errors before training begins.

Training and inference differ. During training, teacher forcing supplies the correct previous target token y_{t-1} to the decoder. During inference, the decoder receives its own previous prediction. Greedy decoding always chooses the most likely next token. Beam search keeps several partial hypotheses, which can improve sequence quality but increases runtime. In applied work, decoding settings should be recorded with the model settings because a translation system with beam size 5 is not being evaluated under the same runtime conditions as a system with greedy decoding.

Attention does not remove every difficulty of RNNs. The encoder and decoder still process tokens sequentially, which limits parallelism during training. Long sequences still require careful batching, masking, and gradient stabilization. However, attention changes the information path. The decoder no longer has to rely on one final encoder state for every target word, and the learned attention weights give a useful diagnostic for alignment mistakes.

10.6 CNNs With Attention

Attention is not tied to recurrent networks. A CNN can produce a grid of image features or a sequence of text features, and an attention mechanism can choose which features are most relevant for the current prediction. This point matters because attention is a general operation over representations, not a special property of translation RNNs.

Image captioning gives the most concrete example. In a basic encoder–decoder captioning system, a CNN maps an image to one vector and an RNN decoder generates the caption [293]. A more informative version keeps a grid of CNN features, such as one feature vector for each spatial region of the image. When the decoder produces the word “dog,” attention can put high weight on the region containing the dog. When it produces “ball,” attention can move to a different region. Xu and colleagues used this idea in an attention-based image-captioning model, connecting visual attention with the attention mechanisms used in machine translation [306].

Figure 10.4 shows the spatial version of the same query, key, value idea. The regions are not pixels by this point; they are CNN feature vectors that preserve coarse location information.

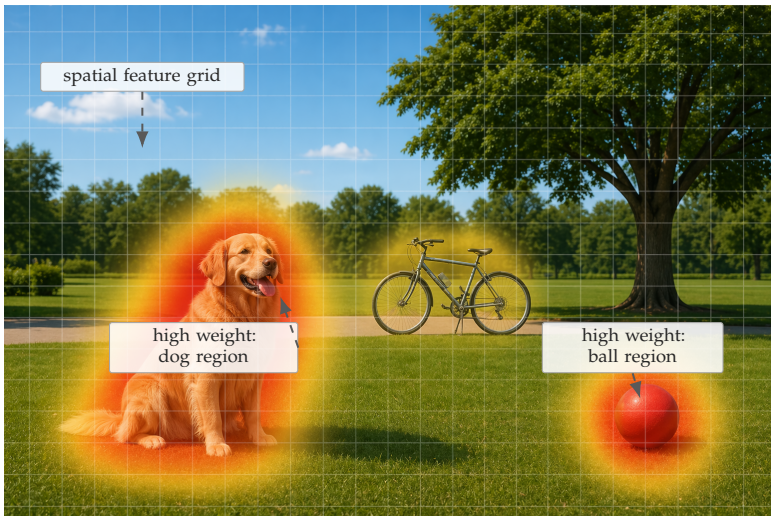


Figure 10.4: AI-generated conceptual illustration of CNN attention for image captioning. The faint grid represents spatial feature regions, and the warm overlays suggest how attention can place higher weight on the dog or ball when those regions are useful for the current generated word. The image is synthetic and is not a dataset example or a measured attention map.

The same principle can be used for text. A one-dimensional convolution can read neighboring tokens and produce contextual features for each position. A decoder can then attend to those convolutional encoder features. The Facebook AI Research paper *Convolutional Sequence to Sequence Learning* developed an architecture for machine translation based entirely on convolutional neural networks, with attention modules in the decoder layers [65]. At a target position t and decoder layer ℓ, the convolutional decoder has a hidden vector $h_t^{(\ell)}$. This vector can serve as the query. The convolutional encoder has produced source-position vectors $s_1, \ldots, s_{T_x}$. After learned projections, these source vectors become keys k_i and values v_i. A dot-product attention module can compute

$$e_{t,i}^{(\ell)} = \left(q_t^{(\ell)}\right)^\top k_i, \qquad \alpha_{t,i}^{(\ell)} = \frac{\exp(e_{t,i}^{(\ell)})}{\sum_{r=1}^{T_x} \exp(e_{t,r}^{(\ell)})}, \qquad c_t^{(\ell)} = \sum_{i=1}^{T_x} \alpha_{t,i}^{(\ell)} v_i.$$

The context vector $c_t^{(\ell)}$ is then combined with the decoder state before predicting the next target token or passing information to the next decoder layer. This is still encoder–decoder attention: the decoder is asking which source positions matter for its current target position. The difference from the RNN case is how the query vectors are produced. During teacher-forced training, causal convolutions can produce decoder states for many target positions in parallel, while an RNN decoder must update its hidden state one step at a time.

The paper is useful in an applied deep-learning course because it framed the architecture not only as a different model family but also as a training-speed improvement. Convolutions over all positions can be computed in parallel during training, while recurrent models must step through positions in order.

The tradeoff is not one-dimensional. An RNN carries a built-in ordered processing bias and handles variable-length sequences naturally. A CNN only sees a local window in each layer, so long-range information has to travel through stacked layers; in return, the convolutions expose much more parallel work to the GPU. Residual connections and gated linear units also helped the Gehring et al. system optimize deep convolutional sequence models. On the translation tasks studied in that paper, the result was strong accuracy and much faster training than a deep LSTM baseline [65]. The lesson is narrower than "CNNs win": model choice has to account for accuracy, parallelism, optimization behavior, and runtime together.

This CNN history also makes the Transformer transition easier to understand. The field was already looking for models that could avoid purely sequential RNN computation while still allowing a decoder to consult the relevant parts of the input. Convolutional sequence models kept attention and replaced recurrence with convolutions. Transformers went further: they made attention itself the main operation for mixing information across positions.

10.7 Self-Attention

Encoder–decoder attention uses a decoder query to retrieve information from an encoder memory. Self-attention uses the same attention idea inside one sequence. Each token creates a query, a key, and a value from its own current representation. The token's new representation is a weighted average of value vectors from the same sequence. In one self-attention layer, every token can receive information from every other token, subject to masks.

This is the central conceptual move from recurrent sequence models to Transformers. An RNN passes information through a chain of hidden states, so token 5 can affect token 20 only through the intermediate states. A self-attention layer instead forms all pairwise token-to-token reading weights in one matrix. Those pairwise reads are still constrained by masks, but they do not require a recurrent hidden-state chain during training.

Definition 10.7.1 (Self-attention) *Self-attention is attention in which the queries, keys, and values are all computed from the same input sequence. If $X \in \mathbb{R}^{T \times d}$ contains one d-dimensional representation for each of T tokens, then a single scaled dot-product self-attention head computes*

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V,$$

and

$$\mathrm{SelfAttn}(X) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

Return to the sentence "the movie was not good." In an early layer, the token "good" may attend strongly to "not," so its representation can encode that the phrase is negative rather than positive. The token "movie" may

attend to nearby content words, while a punctuation token may receive little useful attention. These attention patterns are learned from the training objective. They are not manually specified grammar rules.

The shape calculation is central. Suppose a tokenized sentence has $T = 5$ tokens, each represented by a vector of dimension $d = 4$. Let each attention head use query and key dimension $d_k = 3$ and value dimension $d_v = 3$. Then

$$X \in \mathbb{R}^{5\times 4}, \qquad W_Q, W_K, W_V \in \mathbb{R}^{4\times 3}.$$

Thus $Q, K, V \in \mathbb{R}^{5\times 3}$. The product $QK^\top$ has shape 5×5, giving one score for each ordered pair of token positions. After softmax is applied row by row, the result is an attention matrix whose row i says how much token i reads from every token j. Multiplying that matrix by V returns a 5×3 matrix, one new value mixture per token.

Pause and predict

In the shape example above, one head forms a 5×5 attention-score matrix. Predict how many pairwise scores the same head forms if the sequence length doubles to $T = 10$, and predict whether the attention-score cost grows linearly or quadratically with T.

This shape example also explains the main computational cost. A sequence of length T produces T^2 pairwise attention scores per head. With $T = 100$, one head computes 10,000 scores. With $T = 500$, it computes 250,000 scores. Increasing maximum sequence length can improve accuracy when important evidence appears late in a document, but it also increases memory and runtime substantially. This is why maximum sequence length is one of the most important practical knobs in Transformer fine-tuning.

Length is expensive. Full self-attention scales with T^2, so doubling max length can more than double memory pressure.

Self-attention also needs positional information. The attention formula by itself treats the sequence as a set of token vectors; it compares content but does not know whether a token is first, fourth, or last. Transformer models therefore add positional encodings or learned positional embeddings to token representations. Later models use other position mechanisms, but the applied principle is stable: the model must receive some information about order.

There are two common masks. A padding mask prevents attention to positions added only to make examples in a batch have the same length. A causal mask prevents a position from attending to future positions. Causal masking is needed for autoregressive language modeling: when predicting the next word, the model must not read the true future words. Encoder models used for classification, such as BERT-style models, usually use padding masks but not causal masks. Decoder-only language models, such as GPT-style models, use causal masks during language-model training and generation.

Figure 10.5 makes the matrix view explicit. Each row is one token position asking what it may read, and each column is a token position that could be read unless a mask removes it before softmax.

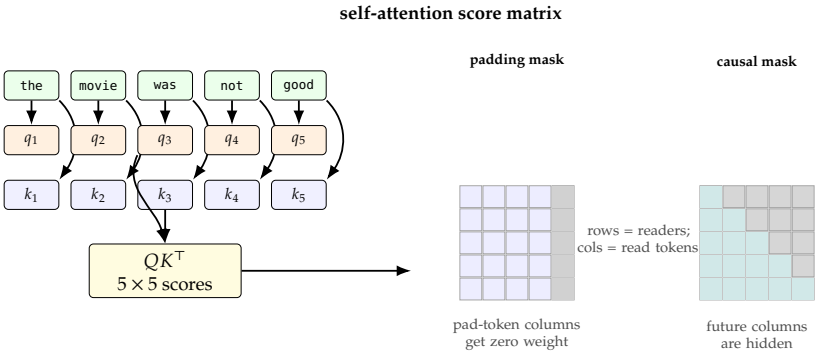


Figure 10.5: A self-attention head forms a score for each ordered pair of token positions. Masks remove invalid reads before the row-wise softmax.

Definition 10.7.2 (Multi-head attention) *Multi-head attention runs several attention heads in parallel, usually with different learned projections W_Q, W_K, W_V. The resulting head outputs are concatenated and projected again. Multiple heads allow the model to represent different kinds of token interactions in the same layer.*

If head m produces output

$$O^{(m)} = \text{Attention}(HW_Q^{(m)}, HW_K^{(m)}, HW_V^{(m)}),$$

then the multi-head output is commonly written as

$$\text{MHA}(H) = [O^{(1)}; \ldots; O^{(M)}]W_O,$$

where the bracket means concatenate along the feature dimension and W_O is the learned output projection.

Multi-head attention should not be interpreted as a promise that head 3 is "the negation head" or head 7 is "the subject head." Individual heads can be interpretable in some analyses, but they can also overlap or carry redundant information. The practical reason for using multiple heads is representational capacity: the model can mix information through several learned similarity spaces at once.

10.8 The Transformer Architecture

The Transformer replaced recurrent sequence processing with attention-based blocks [292]. The original architecture was an encoder–decoder model for sequence transduction, especially machine translation. The encoder read the source sequence using self-attention. The decoder generated the target sequence using masked self-attention and cross-attention to the encoder outputs. Modern Transformer models vary in many details, but the basic components remain central: token embeddings, positional information, multi-head attention, feed-forward networks, residual connections, normalization, and masks.

The selective-retrieval idea from the opening still applies. A Transformer block lets each token ask which other token representations matter now, then uses the feed-forward network to revise the token after that exchange.

Definition 10.8.1 (Transformer block) *A Transformer block is a neural-network module that updates a sequence of token representations using attention and a position-wise feed-forward network, with residual connections and normalization around the sublayers. Encoder blocks use self-attention over the input sequence. Decoder blocks use masked self-attention and, in encoder–decoder Transformers, cross-attention to encoder outputs.*

Three supporting components appear in almost every Transformer block. A residual connection adds a sublayer's input back to its output, which gives deep networks a more direct route for preserving information and gradients. Normalization rescales activations so that training remains more stable. A position-wise feed-forward network is the same small multilayer network applied separately to each token position after attention has mixed information across positions.

In a Transformer encoder block, a token representation first participates in multi-head self-attention. The output is added back to the input through a residual connection, then normalized. A position-wise feed-forward network then applies the same small multilayer network to each position independently, usually expanding the hidden dimension and projecting it back down. A second residual connection and normalization complete the block. The phrase "position-wise" is important: attention mixes information across positions, while the feed-forward network transforms each position's vector after that mixing has occurred.

Two jobs per block. Attention mixes tokens with each other; the feed-forward network transforms each token vector after mixing.

A compact post-normalization sketch is

$$\tilde{H} = \mathrm{LayerNorm}(H + \mathrm{MHA}(H)), \qquad H_{\mathrm{out}} = \mathrm{LayerNorm}\big(\tilde{H} + \mathrm{FFN}(\tilde{H})\big).$$

Many modern implementations use pre-normalization instead, moving LayerNorm before the sublayers, but the same two residual updates remain: attention mixes positions, and the feed-forward network transforms each position.

The decoder block has one extra kind of attention. First, masked self-attention lets each target position read earlier target positions but not future target positions. Second, cross-attention lets the decoder read the encoder outputs. In cross-attention, the decoder states provide the queries, while the encoder states provide the keys and values. For machine translation, this means that each partially generated target position can retrieve relevant source-sentence information. This is the same query-key-value idea used in RNN attention, but implemented inside a deep attention-based architecture.

In practitioner terms, cross-attention is selective retrieval from the encoded source. If a translation model produces the wrong noun, one useful inspection is whether the target position had access to the source token that should have supplied that noun.

Pause and predict

Suppose a translation example has four source tokens and a target prefix of three tokens. Predict which positions are visible to source token 2 in encoder self-attention, to target token 3 in decoder masked self-attention, and to target token 3 in cross-attention.

The original Transformer used sinusoidal positional encodings and scaled dot-product attention [292]. Many later models use learned positional embeddings, relative position information, rotary position embeddings, or other variations. Those details matter for advanced model design, but the first applied reading should track the stable computation: tokens are embedded, position information is injected, attention exchanges information across positions, feed-forward layers transform each mixed representation, and residual connections plus normalization keep the stack trainable.

Figure 10.6 separates those moving parts. The encoder block uses self-attention over the source sequence, while the decoder adds masked self-attention and cross-attention to the encoder outputs.

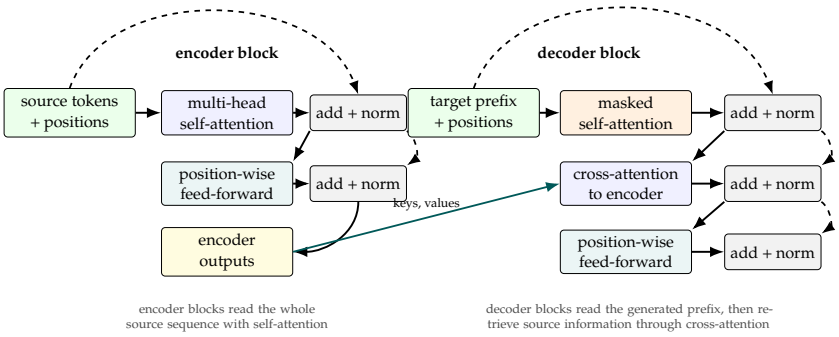


Figure 10.6: Core Transformer block structure. Attention mixes information across positions, feed-forward layers transform each position, and residual plus normalization paths make deep stacks trainable.

The training-speed advantage comes from parallelism. During training, the representations for all source positions can be processed at the same time inside an encoder layer. Similarly, a decoder can process all target positions in parallel during teacher-forced training because the causal mask hides future tokens without requiring a recurrent loop. This is different from an RNN, which must compute hidden states sequentially. The tradeoff is memory: full self-attention stores or computes interactions among all pairs of positions in a sequence.

Inference for autoregressive generation is different from training. A decoder generates tokens one at a time because the next token depends on previously generated tokens. Implementations cache previously computed keys and values so the model does not recompute the entire prefix at every step. Even with caching, generation can be slower than classification because producing a long answer requires many sequential decoding steps. This distinction explains why an encoder model can classify a toxic comment in one forward pass, while a decoder-only language model must generate text token by token.

Training speed is not generation speed. Teacher-forced training can parallelize target positions; autoregressive inference still emits tokens sequentially.

Table 10.2: The same query-key-value definition appears in different parts of a Transformer. Masks determine which positions are visible.

Attention type	Queries come from	Keys and values come from
Encoder self-attention	Source sequence	Source sequence
Decoder masked self-attention	Target prefix	Target prefix
Cross-attention	Decoder states	Encoder outputs

Table 10.2 is a useful diagnostic table when reading code. If a classification model is encoder-only, there is no cross-attention. If a text-generation model is decoder-only, there is no separate source encoder, and the self-attention must be causally masked. If a translation model is encoder–decoder, all three attention types may appear.

10.9 Pretrained Transformer Models

Training a Transformer from scratch usually requires far more data, compute, and engineering care than an introductory applied project can justify. Modern applied NLP therefore often begins with a pretrained checkpoint. The source-task and target-task vocabulary from Chapter 7 applies directly: the checkpoint supplies source-trained parameters, and toxic-comment classification is the target task. In this chapter, the practical choice is whether to use the checkpoint as a frozen feature extractor with a new prediction head, or to fine-tune some or all of its parameters on the target labels.

For classification tasks, encoder-only models are often the natural starting point. BERT is the canonical example: it is a Transformer encoder pretrained with masked language modeling and, in the original formulation, next-sentence prediction [44]. Because a BERT-style encoder can attend bidirectionally over the whole input, it is well suited for tasks where the model should read a complete sentence or document before predicting a label. Toxic-comment classification has this shape. The model reads the comment and then predicts six label logits.

GPT-style models are decoder-only Transformers trained with autoregressive language modeling: predict the next token from previous tokens. The original GPT paper demonstrated that generative pretraining followed by supervised fine-tuning could improve language-understanding tasks [221]. GPT-2 emphasized unsupervised multitask behavior in a larger decoder-only language model [222], and GPT-3 showed strong few-shot performance at much larger scale [17]. For text classification, decoder-only models can be used, but an encoder model is often simpler and faster because it produces a full-input representation in one forward pass without autoregressive generation.

Encoder–decoder pretrained models are useful when the output is itself a new sequence, as in translation, summarization, or question answering cast in a text-to-text format. In these models, the encoder reads the input and the decoder generates the output. This is why the task shape should come before model popularity: toxic-comment classification calls for label logits, translation calls for target tokens, and a chatbot calls for open-ended generation.

Figure 10.7 summarizes this choice. It is a workflow diagram rather than a taxonomy of every checkpoint on the Hub; the first question is what the model must output.

ModernBERT is an example of a more recent encoder-only model family. The ModernBERT paper describes a modernized bidirectional encoder with efficiency and long-context design choices, including an 8192-token native sequence length in the reported models [298]. Compared with the original BERT design, ModernBERT illustrates several updates that now appear often in Transformer engineering: rotary position information rather than the original absolute learned position table, gated feed-forward layers, and a mixture of local and global attention patterns to reduce long-context cost. The details are less important here than the signal

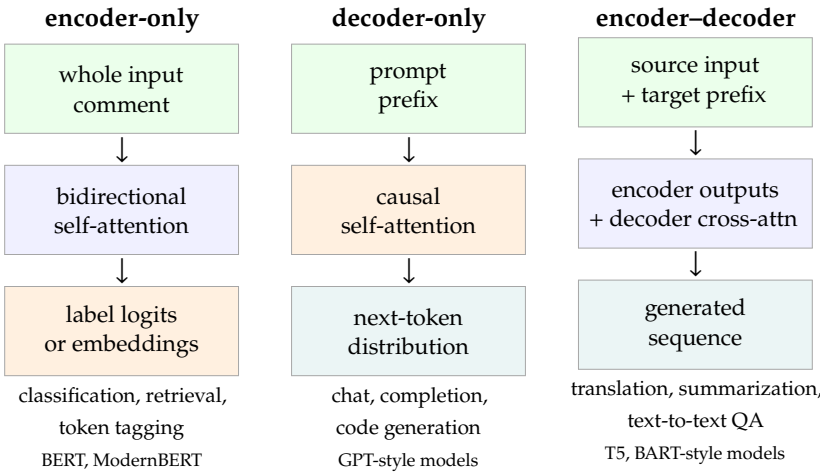


Figure 10.7: Task shape determines which pretrained Transformer family is the natural starting point. Encoder-only models read a full input before prediction, decoder-only models generate the next token from a prefix, and encoder–decoder models condition generation on a separate source input.

Pick the family that matches the output contract before comparing checkpoint popularity.

they send: encoder-only models are still being actively engineered for practical classification and retrieval work, not just preserved as historical predecessors of decoder-only language models.

Tokenization is part of every pretrained Transformer workflow. A tokenizer maps text to token IDs that match the model's pretrained vocabulary. Many Transformer tokenizers use subword units, so an unfamiliar word can be split into pieces rather than replaced by a single unknown token. The tokenizer also adds special tokens, applies truncation, creates attention masks, and sometimes creates token-type IDs. A fine-tuning experiment must record the checkpoint and tokenizer because changing either one changes the meaning of the input IDs.

The applied rule is conservative: start with a small checkpoint and a short maximum sequence length until the pipeline is correct. A BERT-base, DistilBERT, or small ModernBERT checkpoint can reveal tokenization bugs, label-shape mistakes, and metric errors quickly. After the pipeline is correct, larger or more suitable checkpoints can be compared. A larger model that improves validation ROC-AUC, the area under the receiver-operating-characteristic curve, by a small amount but doubles runtime may or may not be the right model, depending on the deployment constraints.

Debug small first. A short, cheap run should prove tokenization, labels, loss, and metrics before expensive comparisons begin.

10.10 The Hugging Face Transformers Library

The Hugging Face Transformers library is a practical interface for using pretrained Transformer models. The library paper describes a unified API for many Transformer architectures and pretrained checkpoints [301]. The online documentation changes over time, so a reproducible experiment should record the installed library version as well as the model checkpoint [112]. For this chapter, memorizing class names matters less than understanding the workflow: tokenize text, create model inputs, choose a pretrained model with a task head, train on a validation-controlled split, and report metrics with runtime.

Figure 10.8 shows that workflow as an experiment pipeline. It keeps the objects that must be recorded for a reproducible run visible instead of hiding them behind a single convenience call.

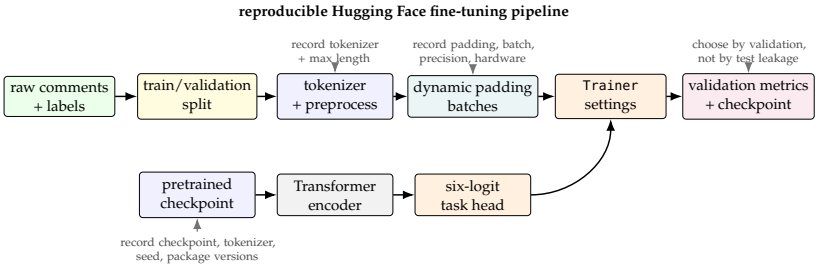


Figure 10.8: A reproducible Hugging Face fine-tuning run makes tokenization, batch construction, model choice, training settings, validation metrics, and checkpoint selection explicit.

The simplest inference interface is a pipeline. A pipeline hides tokenization, model execution, and output formatting behind one call. That abstraction is useful for a quick demonstration, but it hides too much for a serious fine-tuning experiment. A supervised experiment should use the tokenizer, dataset preprocessing, model, training arguments, and metric computation explicitly enough that the run can be reproduced.

The code cells below are written as notebook fragments. They assume that the packages `transformers`, `datasets`, `numpy`, and `scikit-learn` are installed, and that the pretrained checkpoint can be downloaded or has already been cached. In a real assignment notebook, package versions should be printed near the top of the run.

Notebook cell 1

```
from transformers import AutoTokenizer

checkpoint = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(checkpoint)

example = "The movie was not good."
encoded = tokenizer(example, truncation=True, max_length=32)
print(encoded.keys())
print(encoded["input_ids"])
print(encoded["attention_mask"])
```

This first cell is not model training. It is a data check. The `input_ids` are integer token IDs. The `attention_mask` identifies real tokens with 1 and padding tokens with 0, if padding is present. For a toxic-comment classifier, the same tokenizer must be applied to the training, validation, and test splits. A common mistake is to change truncation or padding behavior between splits, which creates an unfair comparison. Another quiet failure is using a checkpoint whose tokenizer splits the domain's important insults, obfuscations, or non-English fragments into unhelpful shards; the model may still train, but the rare-label metrics often show the damage first.

Tokenization is preprocessing. Keep checkpoint, tokenizer, truncation, padding, and max length consistent across splits.

For multi-label classification, each example has a vector of binary labels, not one integer class. If the six labels are toxic, severe toxic, obscene, threat, insult, and identity hate, then a single comment might have label vector

$$y = (1, 0, 1, 0, 1, 0).$$

Pause and predict

Before reading the loss choice, inspect the label vector above. Predict why a softmax over the six outputs would be wrong for this example, and what kind of output nonlinearity should be used when several labels can be true at once.

The model should produce six logits. The loss should be binary cross-entropy with logits, applied independently to each label and then averaged or summed according to the framework convention. A softmax over the six labels would be wrong because the labels are not mutually exclusive.

Multi-label rule. Use six independent sigmoid/BCE decisions, not one softmax over six mutually exclusive classes.

Notebook cell 2

```
from transformers import AutoModelForSequenceClassification

num_labels = 6
model = AutoModelForSequenceClassification.from_pretrained(
    checkpoint,
    num_labels=num_labels,
    problem_type="multi_label_classification",
)
```

The sequence-classification head is a small trainable layer added above the pretrained encoder. The base encoder provides contextual token representations. The classification head turns the representation of the whole sequence into six logits. In many workflows, all parameters are fine-tuned together. When compute is limited, a useful first diagnostic is to freeze the encoder and train only the head. This is usually less accurate than full fine-tuning but can expose label and metric bugs quickly.

A parameter count keeps the word "small" honest. If the pooled encoder representation has width 768 and the toxic-comment task has six labels, a single linear classification head has

$$768 \times 6 + 6 = 4614$$

trainable parameters: one weight vector and one bias for each label. That head is tiny compared with the pretrained encoder, but it is still enough to reveal whether labels, thresholds, and metrics are wired correctly.

The Hugging Face `datasets` library is often used with Transformers, although it is a separate package. It stores columns, maps tokenization functions across examples, caches processed data, and creates framework-ready batches. Dynamic padding is a useful speed setting: instead of padding every example to the global maximum length, a data collator pads each batch only to the longest example in that batch.

Assume that `raw_splits` is a `DatasetDict` with two splits, `"train"` and `"validation"`, and that each split has a text column named `comment_text` plus the six label columns. The homework asks readers to create those splits from the Kaggle CSV files. The mapping step below is the bridge between raw rows and model-ready examples.

Notebook cell 3

```
label_columns = [
    "toxic",
    "severe_toxic",
    "obscene",
    "threat",
    "insult",
    "identity_hate",
]

def preprocess(batch):
    tokenized = tokenizer(
        batch["comment_text"],
        truncation=True,
        max_length=256,
    )
    labels = []
    for row in zip(*(batch[name] for name in label_columns)):
        labels.append([float(value) for value in row])
    tokenized["labels"] = labels
    return tokenized

tokenized_splits = raw_splits.map(
    preprocess,
    batched=True,
    remove_columns=["comment_text"] + label_columns,
)

tokenized_train = tokenized_splits["train"]
tokenized_valid = tokenized_splits["validation"]
```

The `max_length` value in this cell is an experimental choice. A shorter value such as 128 usually trains faster and uses less memory. A longer value such as 256 or 512 may keep evidence that appears late in long comments. The correct choice is empirical: compare validation metrics and runtime under the same split and checkpoint.

For metrics, sigmoid probabilities are used independently for each label:

$$\hat{p}_j = \operatorname{sigmoid}(z_j) = \frac{1}{1 + \exp(-z_j)}, \qquad j = 1, \ldots, 6.$$

A threshold such as 0.5 converts probabilities to predicted labels for F1 scores. ROC-AUC can be computed from probabilities without choosing a fixed threshold, but a rare label may have no positive examples in a small validation split. In that case the per-label ROC-AUC is undefined, so the reporting code must state how the undefined value was handled. The code below reports macro and micro F1, and it reports macro ROC-AUC only when it is defined for the chosen validation split.

A small metric example shows why both metrics are useful. Suppose all label decisions are pooled across a validation batch and the model has 30 true positives, 10 false positives, and 20 false negatives. The micro precision is $30/(30 + 10) = 0.75$, the micro recall is $30/(30 + 20) = 0.60$, and the micro F1 score is

$$\frac{2 \cdot 0.75 \cdot 0.60}{0.75 + 0.60} \approx 0.667.$$

For ROC-AUC, consider one label with two positive and two negative examples. If the positive scores are 0.90 and 0.40, while the negative scores are 0.80 and 0.10, then the positive examples outrank the negative examples in three of the four positive-negative pairs, so the one-label AUC is $3/4 = 0.75$. F1 depends on the chosen threshold; ROC-AUC measures ranking quality before a threshold is fixed.

Notebook cell 4

```
import numpy as np
from sklearn.metrics import f1_score, roc_auc_score

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    probabilities = 1.0 / (1.0 + np.exp(-logits))
    predictions = (probabilities >= 0.5).astype(int)

    metrics = {
        "macro_f1": f1_score(
            labels, predictions, average="macro", zero_division=0
        ),
        "micro_f1": f1_score(
            labels, predictions, average="micro", zero_division=0
        ),
    }
    try:
        metrics["macro_roc_auc"] = roc_auc_score(
            labels, probabilities, average="macro"
        )
    except ValueError:
        metrics["macro_roc_auc"] = float("nan")
    return metrics
```

The `Trainer` API can handle ordinary fine-tuning loops. A serious run should still record the settings. The learning rate, batch size, gradient accumulation, number of epochs, mixed-precision setting, random seed, and evaluation strategy all affect the result. Mixed precision can speed up training on supported CUDA GPUs, but it should be disabled on CPU-only runs or other unsupported hardware.

Notebook cell 5

```
from transformers import DataCollatorWithPadding, Trainer, TrainingArguments

data_collator = DataCollatorWithPadding(tokenizer=tokenizer)
use_fp16 = False  # Set to True only on supported CUDA GPU hardware.

args = TrainingArguments(
    output_dir="toxic-transformer-run",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=32,
    num_train_epochs=3,
    eval_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
    metric_for_best_model="macro_f1",
    greater_is_better=True,
    fp16=use_fp16,
)
```

```
trainer = Trainer(
    model=model,
    args=args,
    train_dataset=tokenized_train,
    eval_dataset=tokenized_valid,
    processing_class=tokenizer,
    data_collator=data_collator,
    compute_metrics=compute_metrics,
)
```

The metric key `"macro_f1"` in `compute_metrics` must match `metric_for_best_model`. The `Trainer` will prefix evaluation metrics with `eval_` in its logs, but the selection setting above names the key returned by `compute_metrics`. Macro F1 averages label-level F1 scores equally; micro F1 pools decisions across labels. Because toxic-comment labels are imbalanced, exact match accuracy alone is not a sufficient metric.

Metric choice matters. Macro F1 exposes rare-label behavior; micro F1 can be dominated by common labels.

Notebook cell 6

```
trainer.train()
validation_metrics = trainer.evaluate()
print(validation_metrics)

pred_output = trainer.predict(tokenized_valid)
validation_probabilities = 1.0 / (
    1.0 + np.exp(-pred_output.predictions)
)
print(validation_probabilities[:2])
```

The final cell starts training, evaluates the selected checkpoint on the validation split, and converts validation logits into probabilities. In a competition workflow, the official test set or public leaderboard should be reserved for the selected model; once it is checked repeatedly, it is no longer independent evidence.

The practical advantage of a library is speed of iteration, not exemption from experimental discipline. Print package versions. Save the checkpoint name. Inspect tokenized examples. Verify the label tensor shape. Check one batch before training. When a run looks too good, inspect per-label counts and a few highest-confidence errors before celebrating; toxic-comment data can reward a model that learns common-label shortcuts while missing rare but important labels. Compare one change at a time.

10.11 A Practical Accuracy and Speed Recipe

Applied deep learning is not only a search for the largest architecture. A useful model is accurate enough for its purpose, trains within available time and memory, and can be evaluated honestly. Transformer fine-tuning makes this tradeoff visible because several settings directly affect both validation quality and runtime.

Start with the checkpoint. A small model trains quickly and is useful for debugging. A larger model may improve validation metrics but can reduce batch size, increase memory use, and slow iteration. A domain checkpoint

can help when its pretraining data resembles the target data, but it can disappoint if the tokenizer, license, or domain assumptions are a poor fit. Treat the checkpoint as an experimental variable, not as a magic constant.

Maximum sequence length is the next expensive choice. The committed toxic-comment reference artifacts compare two otherwise identical DistilBERT settings on a 20,000-example training subset for three epochs:

Setting	Max length	Macro F1	Micro F1	Macro ROC-AUC	Epoch time
A	128	0.4735	0.7776	0.9735	01:15
B	256	0.4659	0.7708	0.9763	01:40

Pause and predict

Before reading the interpretation, predict whether max length 256 should be kept because it improves macro ROC-AUC slightly, or rejected because macro F1, micro F1, and epoch time favor length 128. Name the validation evidence that would change your decision.

The interpretation is not automatic. Here, length 256 improved macro ROC-AUC by about 0.0027 but was slower and had lower macro and micro F1 at the fixed threshold. A first applied choice would keep length 128, save the truncated examples, and only move to 256 if false negatives or threshold analysis show that useful evidence was actually cut off. The saved data-audit summary supports that caution: with `distilbert-base-uncased`, the tokenizer truncated about 19.30% of full training comments at length 128, 6.93% at length 256, and 2.21% at length 512. The task itself is sparse: 16,225 of 159,571 training comments, about 10.17%, have at least one positive label, so macro F1 and per-label failures matter more than a single accuracy-like number. A saved TF-IDF baseline on the same 20,000-example subset reached macro F1 0.5450, micro F1 0.6944, macro ROC-AUC 0.9594, and trained in 4.28s, which is a useful reminder that a neural fine-tune must beat simple baselines on the metric that matters, not just look more modern.

Figure 10.9 gives a compact way to read the measured length-128, length-256, and baseline runs. A point that is both slower and less accurate than another point is dominated; a point that costs more but improves validation behavior requires an applied judgment about whether the gain is worth the cost.

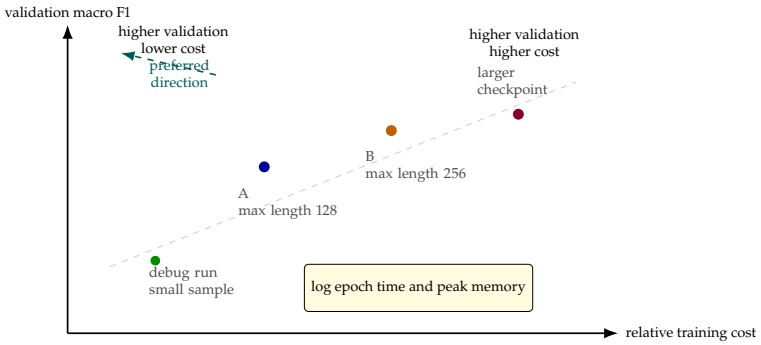


Figure 10.9: Schematic accuracy–cost plot for Transformer fine-tuning. In a completed experiment, each point corresponds to one controlled run with validation quality, epoch time, and peak memory recorded; the useful decision compares validation quality with resource cost, not any quantity alone. The teal arrow marks the preferred direction: higher validation quality at lower relative cost.

Batch construction is another place where speed can disappear. Dynamic padding reduces wasted computation by padding each batch to its own longest example. Bucketing examples by approximate length can reduce waste further, though it adds data-loader complexity. Larger batches can improve hardware utilization, but if the batch does not fit in memory, gradient accumulation can simulate a larger effective batch by accumulating gradients across several smaller batches before an optimizer step.

Numerical precision belongs in the run record. Mixed precision can accelerate training and reduce memory use on GPUs that support it, and it can occasionally affect numerical stability. A run on CPU, a run on an older GPU, and a run on a recent GPU with mixed precision are not comparable as timing measurements unless the hardware and precision are reported.

Optimization settings should be changed deliberately. Fine-tuning BERT-style encoders often uses small learning rates such as 2×10^{-5} or 5×10^{-5}, but the best value depends on the checkpoint, batch size, dataset size, and number of epochs. AdamW is a common optimizer. Learning-rate warmup can make early training more stable. Early stopping can save time when validation metrics stop improving, but it must monitor validation data, not the test set.

Evaluation protocol decides which comparisons are credible. For the toxic-comment homework, model selection should use a validation split. The official Kaggle test labels are not available in the same way as training labels, and repeated leaderboard submissions can become an indirect form of test-set tuning. A report should separate validation evidence from optional leaderboard evidence, and the best validation run should be chosen before any final test or leaderboard result is treated as final.

Validation chooses. The test set or leaderboard should report the selected model, not participate in selecting it.

Before launching a fine-tuning run, make these checks:

- Verify the label columns and confirm that the task is multi-label.
- Print and save package versions, checkpoint name, tokenizer name, and seed.
- Inspect several tokenized examples, including a long example and a short example.
- Start with a small checkpoint or short maximum length until the pipeline is correct.
- Use dynamic padding and record the maximum sequence length.
- Monitor validation loss and task metrics, not training loss alone.
- Record runtime, hardware, precision, batch size, and gradient accumulation.
- Compare one important change at a time.
- Save the best validation checkpoint.
- Evaluate held-out test data only after model selection.

The companion scripts can make this evidence concrete: the data-audit script records label sparsity and truncation rates; the TF-IDF script records a cheap baseline; and the Transformer script writes `comparison_table.csv` when `-save-artifacts` is used. Save representative false positives and false negatives beside those files so a length, threshold, or checkpoint change can be explained by examples rather than by a metric alone.

10.12 Summary

This chapter began with encoder–decoder models. An encoder reads an input object and produces a representation; a decoder uses that representation to produce an output. Early sequence-to-sequence models often passed only one fixed-length vector from encoder to decoder. That design can work on short or simple examples, but it becomes a bottleneck when different output positions need different source details.

Attention solves this problem by making retrieval differentiable. A query is compared with keys, the scores are converted to softmax weights, and the values are averaged with those weights. Additive attention uses a small neural network to compute scores. Multiplicative attention uses dot products or bilinear products and is especially efficient as matrix multiplication. Masks are essential because padding tokens and future tokens should often be invisible.

RNNs with attention were the historical bridge from recurrent sequence models to modern attention-based sequence models. CNNs with attention showed that attention is not tied to recurrence; convolutional sequence-to-sequence models could exploit parallelism while retaining decoder attention over source representations. Self-attention then made attention an internal sequence-mixing operation, allowing each token to read from other tokens in the same sequence.

Deep Transformer networks combine self-attention, encoder–decoder attention, position-wise feed-forward layers, residual connections, normalization, and masks. Encoder-only models such as BERT and ModernBERT fit many classification and retrieval tasks. Decoder-only GPT-style models fit autoregressive generation and become the focus of Chapter 11. Encoder–decoder Transformers are built for sequence-to-sequence generation such as translation and summarization.

For applied work, pretrained models are usually the starting point. The Hugging Face Transformers ecosystem supplies tokenizers, model classes, training utilities, and a large model hub, but it does not remove the need for experimental discipline. Strong results depend on correct labels, appropriate metrics, validation-controlled model selection, sequence-length choices, batching, mixed precision, runtime records, and error analysis.

The practical habit is to treat attention as inspectable retrieval, not as a magic explanation. Ask what each token was allowed to see, what it actually weighted, and whether that evidence supports the reported answer.

10.13 Exercises

Exercise 1. A source sentence has 6 tokens, and each encoder hidden state has dimension 8. The decoder hidden state also has dimension 8. Attention uses the encoder hidden states as both keys and values. Give the shapes of the encoder-state matrix, one decoder query, the score vector, the attention-weight vector, and the context vector.

Exercise 2. Let

$$q = (1, 0), \quad k_1 = (1, 0), \quad k_2 = (0, 1), \quad k_3 = (1, 1),$$

and

$$v_1 = (2, 0), \quad v_2 = (0, 3), \quad v_3 = (1, 1).$$

Using dot-product scores and softmax, compute the attention weights and the context vector to three decimal places.

Exercise 3. A model has unmasked attention scores $(2, 1, -4, 0)$, where the fourth position is padding. Apply a padding mask and compute the masked softmax weights to three decimal places.

Exercise 4. Explain why passing only the final encoder hidden state from an RNN encoder to an RNN decoder can be difficult for a 40-token source sentence. The answer should mention why different output positions may need different source evidence.

Exercise 5. Write the additive-attention score function

$$e_i = w_a^\top \tanh(W_q q + W_k k_i + b_a).$$

Identify which quantities are learned parameters and which quantities depend on the current input example.

Exercise 6. Compare additive attention and dot-product attention. Give one reason additive attention can be flexible, and one reason dot-product attention is fast on modern hardware.

Exercise 7. A self-attention layer receives $X \in \mathbb{R}^{10\times16}$. One attention head uses $W_Q, W_K, W_V \in \mathbb{R}^{16\times4}$. Give the shapes of Q, K, V, the score matrix $QK^\top$, the attention matrix after softmax, and the output of the head.

Exercise 8. Explain the difference between self-attention and cross-attention using a translation example. State where the queries, keys, and values come from in each case.

Exercise 9. For toxic-comment classification, explain why a BERT-style encoder is usually a more direct starting point than a GPT-style decoder-only language model. The answer should mention task shape, bidirectional context, and runtime.

Exercise 10. In the convolutional sequence-to-sequence model of Gehring et al., the decoder state at a target position attends to convolutional encoder states. Explain why this preserves the main benefit of encoder–decoder attention while allowing more parallel training than an RNN decoder.

10.14 Homework: Fine-Tuning a Pretrained Transformer for Toxic Comments

Fine-tune a pretrained Transformer model for multi-label toxic-comment classification using the Jigsaw toxic-comment data from Kaggle [120]. Do not optimize the assignment around a leaderboard number. The submission should show that the data pipeline is correct, the baseline is meaningful, the Transformer run can be repeated, and the accuracy is interpreted together with runtime.

Each example is a text comment with six binary labels: toxic, severe toxic, obscene, threat, insult, and identity hate. Because more than one label can be active for the same comment, this is a multi-label classification problem. The model must produce six logits

$$z = (z_1, \ldots, z_6),$$

and the loss should be binary cross-entropy with logits for the six labels. A softmax over the six labels is not correct because the labels are not mutually exclusive.
The submission must include a non-Transformer baseline and at least one fine-tuned pretrained Transformer. The baseline may be TF-IDF plus logistic regression, a bag-of-embeddings neural model, or another clearly described method that does not use a pretrained Transformer. The Transformer model should use a publicly available checkpoint and tokenizer through a standard library such as Hugging Face Transformers. A small checkpoint is acceptable and often preferred for a first correct run.
All controlled comparisons must use the same train-validation split unless the comparison is explicitly about data splitting. The validation split is used for model selection. If the official Kaggle test or public leaderboard is used, it should be treated as final or optional external evidence, not as a repeatedly queried validation set.

Required Experiments

1. Train and evaluate one non-Transformer baseline.
2. Fine-tune one pretrained Transformer classifier.
3. Run one controlled comparison. Acceptable comparisons include maximum sequence length 128 versus 256, frozen encoder versus full fine-tuning, mixed precision off versus on, or two pretrained checkpoints under the same epoch budget.
4. Perform a short error analysis on false positives and false negatives. Use anonymized, paraphrased, or otherwise appropriate examples if the raw text contains offensive content.
5. For at least one long or ambiguous example, inspect the tokenized and truncated input and state whether the words that support or contradict the label were still visible to the model.

Required Run Table

Report enough fields to reproduce the multi-label Transformer decision: command or notebook cell, random seed, package versions, hardware, checkpoint and tokenizer names, maximum sequence length, truncation rule, threshold rule for turning six sigmoid scores into labels, batch size, gradient accumulation, number of epochs, optimizer, learning rate, scheduler, weight decay, mixed precision setting, number of trainable parameters, best validation loss, validation macro F1, validation micro F1, validation ROC-AUC if computed, runtime, one input-visibility check for a representative error, and whether any final test or leaderboard result was evaluated. If ROC-AUC is reported, state whether it is macro-averaged, micro-averaged, or computed per label. If a label has only one class present in the validation split, its ROC-AUC is undefined; the report must say whether that label was excluded from the average or the split was changed before model selection.

Run	Model or change	Max len.	Scores and time
Baseline	_____	_____	_____
Transformer	_____	_____	_____
Ablation	_____	_____	_____

Written Report

The written report should describe the dataset, label columns, split, and preprocessing; explain the baseline and Transformer model in enough detail for the run to be repeated; present the run table with both accuracy and runtime; and include an error-analysis paragraph. That paragraph should identify at least one concrete

failure pattern, such as ambiguous sarcasm, quoted offensive language, identity terms used non-abusively, very long comments truncated by the tokenizer, or rare labels with too few positive examples. For truncation or long-comment failures, the report should say whether the decisive phrase was inside the maximum sequence length; a modern checkpoint cannot use words that were cut off before inference. When `-save-artifacts` is used, keep the saved false-positive and false-negative CSV files beside the metric table. If raw comments are too sensitive to quote, paraphrase a few representative rows in the report while keeping the artifact path in the run record. The important distinction is whether a failure points to a threshold choice, a sequence-length cut, label sparsity, or a model misunderstanding.

Experimental Result Fields

The following result fields should be filled after running the assignment:

- Baseline validation metrics and runtime: ____.
- Transformer validation metrics and runtime: ____.
- Controlled comparison: ____.
- Error analysis: ____.